Listing 8.31 Integration tests for the WebFlux slice

```
package com.polarbookshop.orderservice.order.web;

import com.polarbookshop.orderservice.order.domain.Order;
import com.polarbookshop.orderservice.order.domain.OrderService;
import com.polarbookshop.orderservice.order.domain.OrderStatus;
import org.junit.jupiter.api.Test;
import reactor.core.publisher.Mono;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.reactive.WebFluxTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.test.web.reactive.server.WebTestClient;
import static org.assertj.core.api.Assertions.assertThat;
import static org.mockito.BDDMockito.given;

@WebFluxTest(OrderController.class)                          ❶
class OrderControllerWebFluxTests {

  @Autowired
  private WebTestClient webClient;                           ❷

  @MockBean                                                  ❸
  private OrderService orderService;

  @Test
  void whenBookNotAvailableThenRejectOrder() {
    var orderRequest = new OrderRequest("1234567890", 3);
    var expectedOrder = OrderService.buildRejectedOrder(
     orderRequest.isbn(), orderRequest.quantity());
    given(orderService.submitOrder(
     orderRequest.isbn(), orderRequest.quantity())
    ).willReturn(Mono.just(expectedOrder));                  ❹

    webClient
      .post()
      .uri("/orders/")
      .bodyValue(orderRequest)
      .exchange()
      .expectStatus().is2xxSuccessful()                      ❺
      .expectBody(Order.class).value(actualOrder -> {
        assertThat(actualOrder).isNotNull();
        assertThat(actualOrder.status()).isEqualTo(OrderStatus.REJECTED);
      });
  }
}
```

❶ Identifies a test class that focuses on Spring WebFlux components, targeting OrderController

❷ A WebClient variant with extra features to make testing RESTful services easier

❸ Adds a mock of OrderService to the Spring application context